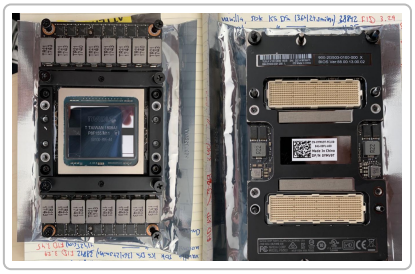
ChatGPT

NVIDIA V100 Private-AI Server Research Report for the Monkey Head Project






Research current through November 11, 2026. This report examines the Tom's Hardware article published November 9, 2026, traces the machine back to the builder's own measurements and code, independently checks the NVIDIA/software facts, and then maps the findings onto Huey and the public `DylanLRPollock/Monkey-Head-Project` repository. The most important distinction is that the lawyer's machine uses **V100-SXM2-32GB modules mounted on four-GPU NVLink baseboards**, whereas the current Monkey Head repository says only that `3 x Tesla V100 32 GB + 1 utility GPU` is a **provisional** direction and explicitly says the exact V100 variants, PCIe topology, acquisition state, power, cooling, framework, and partitioning remain unresolved. Therefore, the two projects use the same GPU generation and memory capacity only if Huey's eventual cards are indeed 32 GB V100s; they should not yet be treated as having the same interconnect architecture. [1]

Executive assessment

The Tom's Hardware story is substantially supported by primary evidence. The builder is **Nick Mermiges**, founder of PLI Labs / Proprietary Legal Intelligence, posting on Reddit as `u/TumbleweedNew6515` and publishing the underlying V100 work through the ProprietaryLegal organization and 1Cat-vLLM community. His November 5 benchmark report gives the exact checkpoint, quantization, serving topology, workload

construction, token-counting method and raw-data repository rather than merely quoting headline throughput. The reported **215 tok/s** result is real in the limited sense that it is a measured single-request, greedy, thinking-off **code-generation** result from DeepSeek-V4.1-Flash-1017 using speculative decoding; it is not the model's general sustained prose rate. Legal text measured about **118 tok/s at 1K context**, falling to about **103 tok/s at 256K**, while code fell from 215 to 160 tok/s over the same context range. [2]

The system is unusually well matched to its model. Eight 32 GB V100s provide 256 GB of physical HBM2 across the machine, but that is **not transparent unified memory**. The builder deliberately treats the hardware as two 128 GB NVLink "islands": model layers 0–19 execute on one four-V100 baseboard and layers 20–39 on the other. Each island uses TP4 internally, while only a pipeline boundary crosses the slower PCIe connection between boards. That PP2×TP4 arrangement is the key architectural lesson for Huey: **partition models according to the real communication topology rather than simply counting total VRAM.** [3]

The lawyer's exact DeepSeek deployment is not a sensible first target for a three-V100 Huey. Its quantized checkpoint is roughly **248.8 GB**, leaving essentially no room for that same all-experts-resident configuration on 96 GB of aggregate V100 memory. More importantly, its performance depends on specialized Volta kernels, an experimental 1Cat-vLLM branch, a two-island SXM2/NVLink topology, DSpark speculative decoding, and an aggressively compressed ~2-bit expert representation. Huey would benefit more immediately from a smaller local language/vision model that keeps safety-critical robot behavior independent of the LLM, with the V100s used for cognition, perception and planning rather than hard real-time actuation. [4]

There is also a real software-lifetime issue. NVIDIA now identifies **R580 as the final driver branch supporting Volta**, while CUDA **12.x is the last CUDA Toolkit family supporting compute capability 7.0**. PyTorch has separately announced that PyTorch **2.14 is the last release receiving official prebuilt CUDA wheels for Volta**, with 2.15+ requiring either an older release or a source build against CUDA 12.x. NVIDIA lists R580 support through June 2028, so V100 remains usable in 2026, but Huey should treat its V100 software environment as a version-pinned appliance rather than casually tracking latest CUDA/PyTorch packages. [5]

My recommended Monkey Head direction is therefore: retain the V100s if they are already available or economically attractive; determine their **exact PCIe/SXM2 identity before designing around them**; establish a reproducible CUDA-12/R580 software image; benchmark each GPU independently; then bring up two- and three-GPU inference with topology-aware model splitting. Do not purchase SXM2 carrier hardware merely to imitate the lawyer's machine unless Huey's mechanical, electrical and cooling design actually benefits from becoming a server-class GPU node. The larger lesson from Mermiges's build is the topology-aware software architecture, not the particular eight-GPU physical implementation. [6]

What the lawyer actually built

The primary November 5 report identifies the production configuration as **eight Tesla V100-SXM2-32GB GPUs**, arranged as **two four-GPU SXM2 baseboards**. Each V100 has 32 GB HBM2. NVIDIA's original V100 specification gives the SXM2 model 5,120 CUDA cores, 640 first-generation Tensor Cores, up to 15.7 FP32 TFLOPS, 125 Tensor TFLOPS, 900 GB/s HBM2 bandwidth, six NVLink connections totaling up to 300 GB/s bidirectional interconnect bandwidth, and a nominal maximum board power of 300 W. [7]

The four GPUs on each builder baseboard are connected through direct NVLink paths. The two four-GPU boards are **not joined by NVLink or NVSwitch**; their cross-board communication goes through PCIe. Each GPU board feeds the host through two PEX/PLX switch cards and SlimSAS cabling. The builder describes four PEX switches total, each serving two GPUs at PCIe Gen3 x8 with each switch given its own x16 CPU-side uplink. PCIe Gen3 delivers about 985 MB/s per lane theoretically, so an x8 endpoint is dramatically narrower than a V100's local HBM2 or aggregate NVLink connectivity. [8]

That hardware should not be confused with NVIDIA's later **NVSwitch** systems. NVSwitch debuted with DGX-2/HGX-2, whose V100 baseboards use six NVSwitch ASICs per eight-GPU baseboard and connect 16 V100 GPUs into a larger fabric. The lawyer's four-GPU carrier boards do not provide that kind of all-GPU NVSwitch fabric; they expose V100 NVLink connections among the local modules and PCIe back to the host. [9]

The reported host is a Gigabyte Threadripper Pro platform with a **12-core Ryzen Threadripper Pro 3945WX**, eight 32 GB DDR4 RDIMMs for **256 GB system RAM**, a single NUMA node from the application's perspective, and two PSUs totaling **2,800 W**. Neither Tom's article nor the November 5 primary post establishes the exact Gigabyte motherboard model, so identifying it more specifically would be speculation. [10]

The builder originally paid roughly **US$1,600 for four 32 GB V100 SXM2 modules with heatsinks plus about US$750 for a board/PEX hardware**, describing the result as approximately **US$2,400 per 128 GB NVLink board**. Two such GPU assemblies therefore cost approximately US$4,800, excluding host CPU, motherboard, RAM, PSUs, cabling, storage and cooling. Tom's Hardware observed that by November 2026 individual used 32 GB SXM2 V100 listings were around US$900, demonstrating how quickly used-market economics can invalidate an old build price. Listing price is not equivalent to actual transaction price. [10]

Using the Bank of Canada's **November 10, 2026 daily indicative rate of US$1 = C$1.4181**, the historical builder figures translate approximately as follows. [11]

Item	Builder-era USD	Approx. CAD at Nov. 10, 2026 FX
Four V100-32GB modules	US$1,600	C$2,269
Board/PEX portion	US$750	C$1,064
Rounded four-GPU assembly	US$2,400	C$3,403
Two four-GPU assemblies	US$4,800	C$6,807
Eight cards at Tom's cited ~US$900 current listing price	US$7,200	C$10,210

Those CAD values are currency conversions, not Canadian retail quotes and not a complete server budget. The Bank of Canada explicitly describes its daily figures as indicative averages. [11]

Power behavior is particularly relevant to Huey. Although an SXM2 V100 is rated at **300 W maximum**, Mermiges now operates the cards at a **150 W power limit** and caps graphics clock at **1,522 MHz**. During

earlier uncapped testing he logged roughly 192–232 W per GPU despite a configured 180 W limit and experienced two fatal PCIe completion-timeout host resets. His hypothesis that a board momentarily stopped responding during power transients remains explicitly **unproven**. A board with a visibly burned power connection was removed, after which stability improved; moving from a 150 W cap to 200 W produced only a reported 0.06 ms-per-round improvement, which he regarded as noise. [12]

At the current 150 W cap, eight GPUs permit up to about **1.2 kW of GPU power alone** before CPU, memory, PEX cards, fans, storage and conversion losses. At NVIDIA's 300 W nameplate limit the eight accelerators alone would represent 2.4 kW. Neither the builder nor Tom's article publishes trustworthy whole-system idle or wall-load wattage, so any claim about total operating consumption beyond those component limits would be an estimate. The appropriate Huey procedure is therefore to measure wall power, rail current and temperature under the actual inference workload rather than derive enclosure requirements from TDP alone. [13]

The machine also exposed reliability issues typical of repurposed datacenter hardware. The builder encountered an uncorrected system-memory ECC fault at DDR4-3200 that disappeared after reducing memory speed to DDR4-2666 and passing repeated 200 GB memory tests. One GPU was still negotiating PCIe x4 instead of x8 at the last reported check. These experiences are valuable because they demonstrate that a multi-V100 machine can "work" while silently operating below its intended topology or with marginal electrical behavior. [12]

The benchmark and what 215 tokens per second actually means

The exact model is **DeepSeek-V4.1-Flash-1017**, not merely generic "DeepSeek V4.1 Flash." DeepSeek describes V4.1-Flash as a multimodal MoE architecture with a 40-layer causal encoder/decoder arrangement, 384 routed experts per MoE layer, six routed experts activated per token, an Engram conditional-memory component and DSpark speculative decoding. DeepSeek's official release describes a 552B-parameter backbone with about 8B active parameters for input processing and 16B for output processing; the 1017 checkpoint is a later weight update using the same basic architecture. [14]

Mermiges did **not** run the original FP8 checkpoint directly. His V100 deployment uses ISTA-DASLab's **GSQ-RCO GGUF quantization**, with `IQ2_XXS` expert weights and `IQ4_NL` Engram data. The builder reports approximately **248.8 GB of quantized model weights**. Every MoE expert needed during decode is kept resident in GPU memory; approximately **110 GB of Engram tables reside in host RAM**. That corrects an earlier statement that all 248.8 GB of model material was entirely in HBM. [15]

The serving stack is a specialized **1Cat-vLLM** Volta/SM70 implementation, using Model Runner V2, full CUDA graphs and **DSpark with five draft tokens per verification round**. The eight GPUs are configured PP2×TP4: four tensor-parallel GPUs operate each pipeline stage. Stage one contains layers 0–19 and stage two layers 20–39. The pipeline implementation deliberately keeps KV/indexer state and Engram-related state local rather than moving those data structures across the stage boundary. [16]

This is why the two-board arrangement performs much better than naïve eight-way tensor parallelism. The high-frequency collectives remain within the four-GPU NVLink island; the expensive inter-board PCIe path carries a pipeline-stage handoff rather than repeated every-layer tensor collectives. Mermiges says an earlier across-all-eight split made performance collapse. This behavior is exactly what one would expect

when communication-sensitive tensor parallelism encounters an interconnect whose effective bandwidth is far lower than the on-board NVLink fabric. [3]

The November 5 methodology is unusually specific: **one request at a time**, maximum configured context 524,288 tokens, thinking disabled, greedy decoding, 1,024 output tokens, prefix cache intentionally bypassed with a random salt, and two measured runs per primary test cell. Decode throughput was calculated client-side as `(tokens − 1) / (time of last token − time of first token)`. First-token time was the median across five content types, while prompt-reading throughput was prompt tokens divided by first-token latency. The largest difference between the paired runs was reported as 2.7 tok/s, and all 45 paired greedy outputs were byte-identical. [17]

Prompt	First token	Code	Structured/JSON	Legal	Prose	Chat
1K	1.22 s	**215.0**	202.5	117.6	89.2	95.1
2K	1.95 s	203.6	207.2	114.8	97.9	94.5
4K	3.26 s	194.9	195.6	111.6	92.9	103.4
8K	6.94 s	207.9	209.0	109.1	92.2	108.2
16K	10.4 s	201.8	212.7	105.6	95.8	110.7
32K	17.2 s	200.5	206.6	116.5	90.1	106.9
64K	30.9 s	194.1	196.3	108.3	92.1	94.7
128K	59.2 s	182.8	196.5	110.9	91.1	95.8
256K	118.7 s	160.0	179.7	103.1	84.1	80.5

These are the builder's November 5 measurements, not independent third-party reproduction. The underlying prompts were real corpora: vLLM Python source for code, LegalBench JSON for structured workloads, public legal text plus LegalBench prompts for legal material, *War and Peace* for prose, and a chat-style task over *Les Misérables*. [17]

The **215 tok/s headline is principally a speculative-decoding result**. DSpark proposes five tokens and the target model verifies them together. Code and structured JSON are more predictable and accepted roughly 3.6–3.8 draft tokens per round, causing about 4.6–4.7 output tokens to emerge from a verification round. Legal/prose/chat accept substantially fewer drafts, so the same engine produces fewer visible tokens per second. The underlying verification round takes roughly 23 ms at short context and around 27 ms at 256K; therefore the large variation between 215 tok/s code and ~90–118 tok/s natural language is mostly a draft-acceptance effect rather than the GPU suddenly doing twice as much matrix math on code. [17]

Long-context prefill behaves very differently from decode. At 128K the builder measured approximately **2,198–2,220 prompt tokens/s**, yielding around 59 seconds to first token. At 256K the rate remained around 2,197–2,222 tok/s but startup time rose to about 119 seconds. The larger model can therefore ingest very large document sets, but "supports long context" should not be confused with conversational immediacy. [17]

The same build was quality-tested at **79.9% MMLU-Pro over 700 selected items**, **147/164 HumanEval+ pass@1**, and **0.716 balanced accuracy across 22 LegalBench tasks**. More importantly, the builder measured fidelity against the original unquantized model and found only **89.0% top-next-token agreement on a general panel and 92.5% on a legal panel**, explicitly attributing much of the difference to the 2-bit expert quantization. Thus "fits in 256 GB" comes with a measurable approximation cost. [18]

A useful reproducibility qualifier is that the principal PP2×TP4 integration was still an **open 1Cat-vLLM pull request** when checked, even though the builder publishes the branch, measurement procedures and raw data. This makes the experiment far more reproducible than a typical social-media benchmark, but it is not equivalent to a stock upstream `pip install vllm` deployment. [19]

V100 capabilities and the 2026 software reality

The Tesla V100 remains an impressive memory-bandwidth device despite its age. A 32 GB SXM2 card supplies 900 GB/s HBM2 bandwidth, while a four-card machine theoretically exposes 3.6 TB/s of aggregate local memory bandwidth and eight cards 7.2 TB/s. Those aggregate figures do **not** mean one kernel can read arbitrary data anywhere at 7.2 TB/s: each GPU owns its local HBM, and inter-GPU traffic is constrained by NVLink or PCIe according to where the data resides. [7]

Volta's Tensor Cores are fundamentally **FP16-oriented**. NVIDIA's current compute-capability tables and TensorRT support matrices show V100 supporting FP16 Tensor Cores but **not INT8 Tensor Cores**. V100 can execute INT8 operations, but it lacks the later architectures' dedicated INT8 Tensor Core acceleration, and it has none of the newer FP8/FP4 Tensor Core capabilities. This is why modern low-bit V100 inference stacks frequently store weights in 2/4/8-bit formats but dequantize them into a datatype or custom operation that Volta can execute efficiently rather than expecting native FP4/INT4 Tensor Core math. [20]

That distinction matters greatly for Huey. "4-bit model" describes **storage and traffic representation**, not necessarily arithmetic precision. A 4-bit checkpoint can save enough VRAM to make a model feasible while still spending substantial compute on FP16 or mixed-format operations. Which quantization performs best on V100 is therefore runtime-specific. vLLM's current hardware matrix lists GPTQ, bitsandbytes, GGUF and DeepSpeedFP as available on Volta while AWQ and Marlin are not supported on that architecture in the standard implementations. [21]

Official vLLM documentation continues to state a minimum NVIDIA compute capability of **7.0**, explicitly naming V100, but that broad compatibility statement should not be taken to mean every new model or every quantization kernel works on Volta. Mermiges's DeepSeek work exists precisely because modern architectures frequently require dedicated SM70 kernels and patches. [22]

`llama.cpp` remains another practical V100 path. Its current CUDA build configuration still includes architecture **70** when compiling with pre-CUDA-13 toolchains, and its documentation supports explicitly setting `CMAKE_CUDA_ARCHITECTURES=70`. Current release builds still include CUDA 12 packages, although individual V100 regressions have been reported with some CUDA 12.x revisions; version-pinning and local validation are therefore prudent. [23]

The larger compatibility cliff is now clearly documented by NVIDIA. Volta compute capability 7.0 debuted with CUDA 9.0 and ends with **CUDA 12.x**; the **R580** driver branch is the final driver line supporting it.

NVIDIA's driver lifecycle table gives R580 an end-of-life date of June 2028. That does not mean V100 stops working in 2028, but mainstream new-driver/new-toolkit compatibility ceases to advance past that boundary. [24]

PyTorch adds another constraint: the project announced that **PyTorch 2.14 is the last release with prebuilt Volta-compatible CUDA wheels**. Starting with 2.15, users needing V100 must remain on 2.14 or compile from source against CUDA 12.x. For an appliance such as Huey this is manageable if the inference environment is containerized and pinned; it is problematic only if the project expects indefinitely installing the newest binary packages without compatibility engineering. [25]

A reasonable Huey baseline in late 2026 is therefore an R580 driver with a known-good CUDA 12.x environment, plus separately frozen inference environments for `llama.cpp` and the chosen PyTorch/vLLM stack. The environment should be treated as part of the robot's configuration evidence, including driver version, CUDA toolkit, runtime commit, GPU firmware/VBIOS identifiers and quantized-model hashes. That approach aligns closely with Monkey Head's existing emphasis on checksums, explicit evidence, rollback and reproducibility. [26]

Direct implications for Huey

The public Monkey Head repository currently defines Huey as one embodied AI node containing Brain functions, Body functions and a local compute kernel, with optional Farm infrastructure rather than making remote compute mandatory. It specifically calls for operator-visible state, attributable evidence, safe-stop/recovery behavior and useful standalone operation. This is an excellent match for private local inference: the V100 subsystem can be the high-level cognition accelerator without making Huey dependent on an Internet AI service. [27]

However, the repo's hardware state is deliberately provisional. Huey V4 currently names an existing Intel Core i9 / ASUS TUF / Optane system as the intended kernel after validation, with **four node-local accelerators as a working direction and** `3 x Tesla V100 32 GB + 1 utility GPU` **only provisional**. It explicitly says exact variants, PCIe topology, power, cooling, retention and service access remain unresolved, and explicitly warns that aggregate accelerator memory must not be described as transparent unified VRAM. That is exactly the correct caveat to preserve after studying the lawyer's system. [28]

If Huey's three V100s turn out to be **PCIe V100-32GB cards**, the lawyer's SXM2 performance cannot simply be extrapolated. NVIDIA rates V100 PCIe host interconnect at PCIe Gen3 while the SXM2 part can use up to 300 GB/s aggregate bidirectional NVLink. Three PCIe cards may still support CUDA peer-to-peer depending on motherboard/root-complex layout, but heavy tensor-parallel collectives can become communication-bound. The first physical diagnostic should therefore be `nvidia-smi topo -m`, followed by CUDA's `p2pBandwidthLatencyTest`, before choosing TP3 or another partitioning scheme. [7]

For three 32 GB cards Huey has **96 GB of aggregate physical HBM**, but the useful capacity depends on partitioning. A model split by layers can use nearly all three cards because each device stores different layers. Tensor parallelism can similarly shard weights but usually requires more frequent inter-GPU communication. Independent workloads may instead assign speech or vision to one GPU and language

inference to the others. There is no universal “96 GB GPU” abstraction here, which is why the repository’s existing wording is technically sound. [29]

For a robot, I would initially favor **workload separation over heroic frontier-model packing**. One possible allocation is two V100s for the primary LLM/VLM and the third for vision, embeddings, auxiliary models or a second smaller language service, while the utility GPU handles display/camera duties if necessary. A different configuration may prove better after benchmarking, but this arrangement reduces interference between continuous perception and bursty language generation and leaves simpler failure boundaries than an always-synchronized TP3 cognition stack. This is an engineering recommendation rather than a claim that the existing Huey implementation already works this way. The repository presently leaves workload allocation unresolved. [30]

Speech is particularly suitable for local operation. Huey’s current V1 proof path already starts from a known MP3 fixture, performs local transcription, then currently routes cognition through a selected API-backed path and records the result in a structured log. Replacing that cognition hop with a local OpenAI-compatible endpoint is therefore conceptually small compared with redesigning the whole robot. The V100 server can expose an API that PyHuey/HueyOS calls through localhost or a private LAN address, while keeping an API/cloud route available only as an explicitly selected fallback. [31]

Vision is also increasingly feasible. DeepSeek-V4.1-Flash itself is multimodal, and V100-targeted community implementations have demonstrated contemporary vision-language models on Volta, but Huey does not need a 552B-class model merely to interpret cameras. A small or mid-size dedicated VLM can deliver substantially lower first-token latency and reserve memory for continuous operation. The correct target should be chosen through Huey-specific camera workloads—object identification, panel reading, workspace interpretation, person/gesture detection and OCR-like tasks—rather than model leaderboard size alone. [32]

The strongest safety design is already implicit in the project repository: **LLM output must not directly actuate hardware**. Monkey Head explicitly says that an authenticated controller command remains a request and must pass authorization, validation, operator confirmation where appropriate, safe-stop logic and Body execution gates; HIMS transport success similarly must not automatically become physical action. The same boundary should apply to an LLM. The model may propose “rotate left 20 degrees,” but deterministic Body software should validate range, collision state, current operating mode, emergency-stop condition and actuator limits before anything moves. [33]

A strong Huey control stack would therefore separate **cognition latency** from **control latency**. Speech/LLM planning can tolerate hundreds of milliseconds or several seconds; motor watchdogs, current limits, obstacle stop logic and emergency-stop behavior cannot. The latter should continue functioning if the V100 process hangs, CUDA crashes, the model produces malformed JSON or every GPU disappears from the PCIe bus. That recommendation follows directly from both the project’s authority model and the lawyer system’s real experience with host resets and PCIe faults. [34]

Privacy architecture is another strong overlap. The lawyer’s entire motivation is keeping confidential legal files on-premises; Huey can apply the same principle to microphone audio, camera frames, household information, memory stores and operator conversations. HIMS and PyHuey can remain inside a local authenticated network, with outbound cloud access isolated behind an explicit gateway rather than letting

every inference component reach the Internet. This extends the project's current privacy/no-secret and authenticated-message direction rather than replacing it. [35]

Implementation priorities, tests, costs and risk register

The immediate priority is **hardware truth**, because nearly every architecture choice follows from it. Before model work, record the exact V100 part numbers, PCIe or SXM2 form factor, VBIOS, memory size, ECC state, PCIe negotiated width/speed, motherboard model, PSU model/rail arrangement, connector type, cooling design and slot/root-complex topology. The current repository explicitly says these items are unresolved, so converting them into a signed hardware inventory would advance Monkey Head's own evidence model as well as the AI work. [30]

A practical acceptance sequence is: run each V100 individually under ECC/memory stress; inspect `nvidia-smi -q`; test GPU memory; run sustained FP16 GEMM and a real inference load; inspect PCIe error counters; then test P2P bandwidth/latency for every GPU pair. Repeat at cold start and after thermal soak. Any card with rising ECC counts, PCIe retraining, connector discoloration or thermal instability should fail acceptance until understood. The builder's burned connector, x4 link and ECC incidents demonstrate why these tests are not administrative overhead. [12]

For software, build one **known-good frozen Volta image** first rather than immediately optimizing. An appropriate experimental matrix would include R580 + CUDA 12.x, `llama.cpp` compiled explicitly for `sm_70`, a PyTorch 2.14 environment, stock vLLM where supported, and 1Cat-vLLM only for models that actually benefit from its Volta paths. Keep engine versions and model hashes in Huey's evidence/logging system. [36]

A minimal `llama.cpp` V100 build remains straightforward:

```
cmake -B build \
  -DGGML_CUDA=ON \
  -DCMAKE_CUDA_ARCHITECTURES=70 \
  -DCMAKE_BUILD_TYPE=Release

cmake --build build -j
```

That architecture setting is directly supported by the current `llama.cpp` CUDA build logic when using CUDA 12-era toolchains. [37]

Huey's benchmark suite should emphasize **robot behavior rather than peak tok/s**. Record time-to-first-token for 0.5K, 4K, 16K and 64K prompts; decode tok/s for conversational prose and structured JSON separately; simultaneous speech+vision+LLM latency; tool-call validity; memory usage per GPU; PCIe/NVLink traffic; GPU power; wall power; temperatures; ECC changes; recovery after killing a worker; recovery after resetting a GPU; and latency while another accelerator is saturated. Mermiges's results demonstrate why workload type and first-token latency must be reported separately from maximum speculative decode throughput. [17]

For Huey, a sensible interactive target is not “beat 215 tok/s.” Even 30–60 tok/s is faster than a human can comfortably consume as conversational text; what matters more is consistently low startup latency, timely tool calls, perception responsiveness, deterministic safety behavior and fault recovery. The lawyer’s 128K example shows the point vividly: **183 tok/s code decode is impressive, yet the user waits nearly a minute before the first token.** [17]

The most relevant incremental upgrade is therefore often **interconnect/topology improvement**, not more nominal FLOPS. If Huey’s cards are PCIe V100s, make sure they negotiate full intended width and have P2P access before adding a fourth card. If they are SXM2 modules not yet integrated, a validated four-GPU carrier can create the same type of local NVLink island used in Mermiges’s project, but it also adds unusual power-delivery, cooling, cabling and mechanical requirements. The Tom’s build proves such systems can work; it equally demonstrates that they are closer to custom server engineering than normal desktop GPU installation. [38]

A fourth 32 GB V100 would bring Huey from 96 to **128 GB aggregate HBM** and could be disproportionately useful if it produces a balanced TP4 topology. Community 1Cat-vLLM measurements show modern 27–35B models operating extremely quickly on four V100s, including several measured 100–200+ tok/s speculative configurations, although those figures use highly optimized SM70 kernels and should not be assumed for Huey before replication. [39]

Moving beyond V100, used A100-class hardware would provide newer Tensor Core precision modes, better software longevity and stronger modern-inference support; newer workstation/datacenter GPUs can add much larger per-card memory and native FP8/FP4-class arithmetic depending on generation. The disadvantage is acquisition cost and, in some configurations, power or form-factor complexity. For a project that already owns V100s, the economic comparison should be **incremental cost to make the V100 system reliable** versus replacing working memory capacity, not V100’s original MSRP versus a new GPU’s MSRP. NVIDIA’s own compatibility matrix makes clear, however, that Ampere remains supported by current CUDA while Volta has entered its final-toolchain window. [40]

The project risk register is therefore:

Risk	Likelihood / impact	Evidence	Mitigation
Exact Huey V100 variant differs from lawyer’s SXM2 hardware	High / High	Repo leaves variant unresolved. [30]	Inventory first; do not design around NVLink until measured.
PCIe topology throttles tensor parallelism	Medium–High / High	Lawyer avoided cross-board TP because PCIe was the bottleneck. [41]	`nvidia-smi topo -m`, P2P tests, layer/pipeline splitting.
Volta software ecosystem ages out	Certain / Medium–High	CUDA 12.x and R580 are final Volta lines. [24]	Freeze images; source-build where necessary; retain rollback packages.

Risk	Likelihood / impact	Evidence	Mitigation
Modern quantization kernels are unavailable on SM70	High / Medium	V100 lacks modern low-bit Tensor Core modes; vLLM support varies by quantizer. [42]	Benchmark GGUF/GPTQ/ custom paths rather than assuming AWQ/FP8 performance.
Used GPU ECC or physical failure	Medium / High	Builder experienced RAM ECC faults, burned connector and link-width issue. [12]	Burn-in, ECC monitoring, connector inspection, spare strategy.
Thermal/power instability	Medium / High	Eight-card system required power/clock limiting. [12]	Power caps, rail instrumentation, thermal soak tests and headroom.
LLM failure affects physical safety	Potentially severe	Huey already defines actuation authority boundaries. [31]	Keep deterministic Body safety outside inference process.
Very large context harms responsiveness	High / Medium	128K ≈59 s TTFT; 256K ≈119 s in benchmark. [17]	Retrieval, summarization and bounded working context instead of stuffing history.
Low-bit model quality regression	Medium / Medium–High	2-bit build only 89% general next-token agreement with original. [17]	Compare quantizations on Huey task suite, not tokens/s alone.
Cloud fallback leaks private sensor/history data	Design-dependent / High	Local privacy is central to lawyer system and Monkey Head boundaries. [43]	Local-first gateway, explicit consent/routing, redact or prohibit outbound classes.

My prioritized implementation path for Monkey Head is therefore **inventory → electrical/thermal acceptance → topology measurement → frozen Volta software image → one-GPU baseline → multi-GPU topology experiments → local Huey cognition endpoint → speech/vision integration → safety/ failure testing → optimization**. Only after those steps should the project consider reproducing exotic 1Cat-vLLM DeepSeek configurations. That order minimizes the chance of spending weeks optimizing a model around an incorrect assumption about the actual V100 hardware. [44]

For an initial Huey deployment, I would keep the robot-local stack modest: continuous local transcription and perception, a responsive quantized language or vision-language model, retrieval over Huey's structured memory, and deterministic Body services behind HIMS. A larger V100 server or future Farm could handle long-document reasoning, code generation, offline indexing or heavyweight planning without becoming necessary for safe movement or basic interaction. That separation closely matches the repository's distinction between one valid local Huey node and optional shared Farm infrastructure. [45]

Evidence, assumptions and primary sources

Finding	Evidence status
Builder is using 8× V100-SXM2-32GB	**Confirmed by builder, Tom's and public code/discussion.** [46]
Two four-GPU NVLink boards, PCIe between boards	**Confirmed by builder.** [47]
215 tok/s	**Confirmed as builder-measured 1K-context, thinking-off, greedy code throughput; not general model speed.** [17]
~118 tok/s legal writing	**Confirmed as builder measurement at short context; falls with context length.** [17]
DeepSeek-V4.1-Flash-1017	**Confirmed by primary benchmark publication.** [17]
GSQ-RCO / IQ2_XXS experts / IQ4_NL Engram	**Confirmed by builder.** [17]
248.8 GB quantized model	**Builder-reported; consistent across the primary post and article.** [48]
110 GB Engram tables in CPU RAM	**Builder correction; not independently instrumented by this report.** [12]
PCIe resets were caused by power spikes	**Unverified hypothesis. Builder explicitly says not proven.** [12]
150 W is optimal for all V100 workloads	**Not established.** It was neutral versus 200 W in this particular model/build. [12]
Current Huey owns/has installed three V100s	**Not established by the public repo.** The public repository explicitly says V100 acquisition/installation is not claimed and the 3× V100 configuration remains provisional. [49]
Huey V100s are SXM2	**Unverified.** Public repository does not state the variant. [30]
Huey has 96 GB usable as one transparent memory pool	**False as stated.** Three 32 GB cards provide 96 GB aggregate physical memory, but the repo correctly rejects transparent-unified-VRAM wording. [30]

Key direct source URLs, included because the project requested reproducible primary references:

Source	Direct URL
Tom's Hardware article	`https://www.tomshardware.com/tech-industry/artificial-intelligence/lawyer-builds-private-ai-server-with-eight-used-nvidia-v100-gpus-up-to-215-tokens-per-second-to-keep-legal-files-offline-costing-about-usd2-400-per-four-gpu-board`
Builder Reddit update	`https://www.reddit.com/r/homelab/comments/1xoumr9/lawyers_v100_homelab_update_two_4card_sxm2_nvlink/`
Builder's 1Cat-vLLM benchmark discussion	`https://github.com/1CatAI/1Cat-vLLM/discussions/1378`
PP2×TP4 implementation PR	`https://github.com/1CatAI/1Cat-vLLM/pull/1370`
Builder's V4.1/V100 measurement repository	`https://github.com/ProprietaryLegal/Volta-sPLIt-DeepSeek-V4.1-Flash-1017-8xV100-SXM2-PP2xTP4`
PLI V100 DeepSeek repository	`https://github.com/ProprietaryLegal/deepseek-v4-flash-v100`
1Cat-vLLM	`https://github.com/1CatAI/1Cat-vLLM`
DeepSeek-V4.1-Flash model	`https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash`
DeepSeek-V4.1-Flash-1017	`https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash-1017`
NVIDIA V100 datasheet	`https://images.nvidia.com/content/technologies/volta/pdf/tesla-volta-v100-datasheet-letter-fnl-web.pdf`
NVIDIA CUDA architecture lifecycle matrix	`https://docs.nvidia.com/datacenter/tesla/drivers/cuda-toolkit-driver-and-architecture-matrix.html`
NVIDIA driver support lifecycle	`https://docs.nvidia.com/datacenter/tesla/drivers/latest/supported-drivers-and-cuda-toolkit-versions.html`
llama.cpp	`https://github.com/ggml-org/llama.cpp`
Monkey Head Project	`https://github.com/DylanLRPollock/Monkey-Head-Project`

The central conclusion for Monkey Head is therefore quite favorable to the V100 strategy, but with an important qualification: **the article does not prove that "three V100s will make Huey a 215-tok/s**

DeepSeek system.” What it proves is more useful. It demonstrates that Volta remains capable of surprisingly strong modern inference when memory placement, quantization, speculative decoding, topology and power are engineered together; it demonstrates that **topology is as important as aggregate VRAM**; and it provides a real public example of private, offline AI operating on hardware of the same GPU family Huey is considering. The most valuable thing to copy is Mermiges’s measurement discipline and topology-aware partitioning—not the assumption that eight SXM2 V100s are the right physical architecture for the robot. [50]

---

[1] Lawyer builds private AI server with eight used Nvidia V100 GPUs — up to 215 tokens per second to keep legal files offline, costing about $2,400 per four-GPU board | Tom's Hardware
https://www.tomshardware.com/tech-industry/artificial-intelligence/lawyer-builds-private-ai-server-with-eight-used-nvidia-v100-gpus-up-to-215-tokens-per-second-to-keep-legal-files-offline-costing-about-usd2-400-per-four-gpu-board?type=AI&utm_source=chatgpt.com

[2] [3] [8] [10] [12] [35] [38] [43] [47] [48] Lawyer's V100 homelab, update: two 4-card SXM2 NVLink boards, four PEX switches, SlimSAS everywhere, one burnt power connector, and a 250 GB model running entirely in VRAM at 200+ tok/s : r/homelab
https://www.reddit.com/r/homelab/comments/1xoumr9/lawyers_v100_homelab_update_two_4card_sxm2_nvlink/

[4] [15] [16] [17] [18] [46] [50] DeepSeek-V4.1-Flash-1017 on 8x V100 (two boards, PP2 x TP4): ~200 tok/s code, 2,200 tok/s prompt reading at 128K · 1CatAI 1Cat-vLLM · Discussion #1378 · GitHub
https://github.com/1CatAI/1Cat-vLLM/discussions/1378

[5] [24] [26] [36] [40] CUDA Toolkit, Driver, and Architecture Matrix — NVIDIA Data Center Drivers
https://docs.nvidia.com/datacenter/tesla/drivers/cuda-toolkit-driver-and-architecture-matrix.html?utm_source=chatgpt.com

[6] [27] [28] [29] [30] [31] [33] [34] [44] [45] GitHub - DylanLRPollock/Monkey-Head-Project: Huey is a prototype robotic A.I./O.S. under the Monkey-Head-Project · GitHub
https://github.com/DylanLRPollock/Monkey-Head-Project

[7] [13] NVIDIA TESLA V100 GPU ACCELERATOR
https://images.nvidia.com/content/technologies/volta/pdf/tesla-volta-v100-datasheet-letter-fnl-web.pdf?ncid=so-othe-69398&utm_source=chatgpt.com

[9] NVIDIA Fabric Manager — NVIDIA Fabric Manager
https://docs.nvidia.com/hgx-platforms/fabric-manager-user-guide/index.html?utm_source=chatgpt.com

[11] Daily exchange rates - Bank of Canada
https://www.bankofcanada.ca/rates/exchange/daily-exchange-rates/?utm_source=chatgpt.com

[14] [32] deepseek-ai/DeepSeek-V4.1-Flash · Hugging Face
https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash?inference_provider=fireworks-ai&utm_source=chatgpt.com

[19] [41] [Model][SM70] DeepSeek-V4.1: PP2 x TP4 for the #1277 package (8 V100s) by ProprietaryLegal · Pull Request #1370 · 1CatAI/1Cat-vLLM · GitHub
https://github.com/1CatAI/1Cat-vLLM/pull/1370

[20] [42] Abstract
https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-861/support-matrix/index.html?utm_source=chatgpt.com

[21] Quantization - vLLM
https://docs.vllm.ai/en/latest/features/quantization/?utm_source=chatgpt.com

22 GPU - vLLM
https://docs.vllm.ai/en/v0.11.1/getting_started/installation/gpu/?utm_source=chatgpt.com

23 37 llama.cpp/ggml/src/ggml-cuda/CMakeLists.txt at master · ggml-org/llama.cpp · GitHub
https://github.com/ggml-org/llama.cpp/blob/master/ggml/src/ggml-cuda/CMakeLists.txt?utm_source=chatgpt.com

25 Notice: CUDA 12.6 wheels will no longer be published from PyTorch 2.15 (drops Maxwell / Pascal / Volta) - release/packaging - PyTorch Developer Mailing List
https://dev-discuss.pytorch.org/t/notice-cuda-12-6-wheels-will-no-longer-be-published-from-pytorch-2-15-drops-maxwell-pascal-volta/3432?utm_source=chatgpt.com

39 GitHub - 1CatAI/1Cat-vLLM: V100 / SM70-focused vLLM engineering fork for modern LLM inference. · GitHub
https://github.com/1CatAI/1Cat-vLLM

49 GitHub - DylanLRPollock/Monkey-Head-Project: Huey is a prototype robotic A.I./O.S. under the Monkey-Head-Project · GitHub
https://github.com/DylanLRPollock/Monkey-Head-Project?utm_source=chatgpt.com